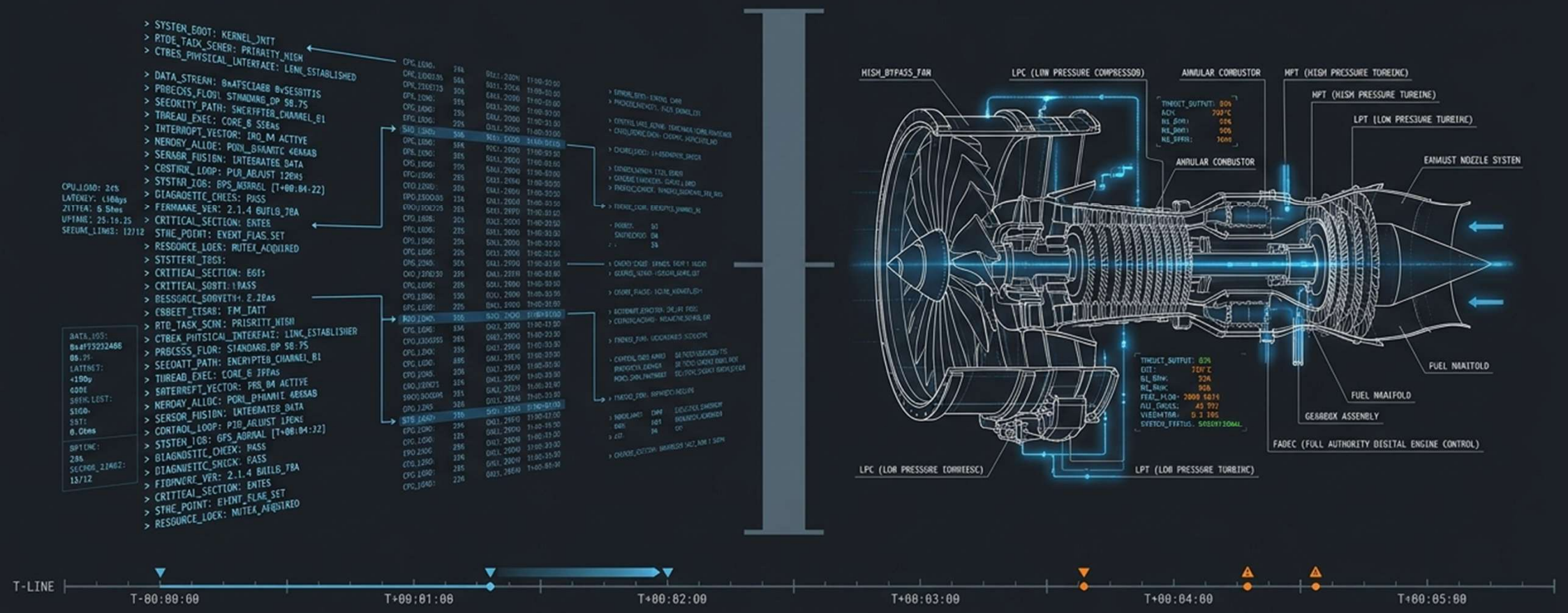
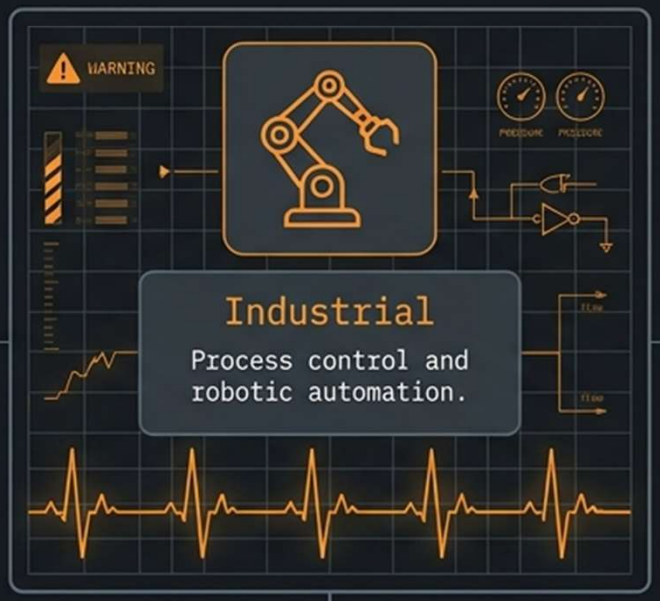
THE ENGINE OF THE PHYSICAL WORLD

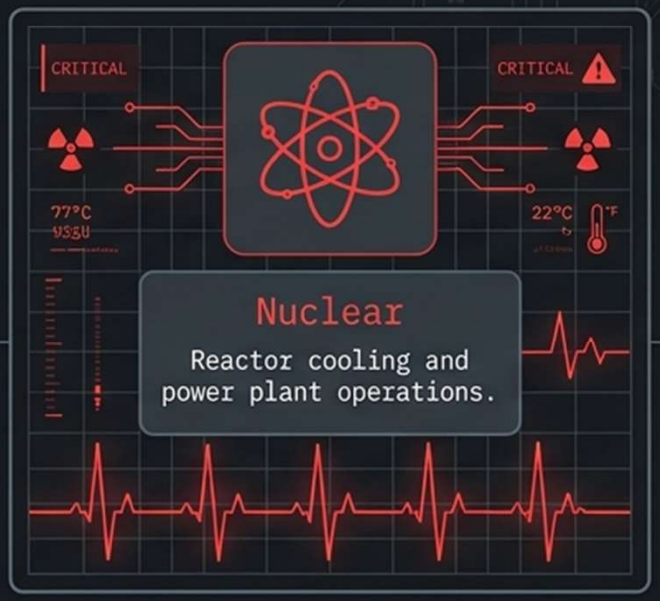
Real-Time Operating Systems for Cyber-Physical Systems



NotebookLM

In the physical world, a missed deadline is a disaster.



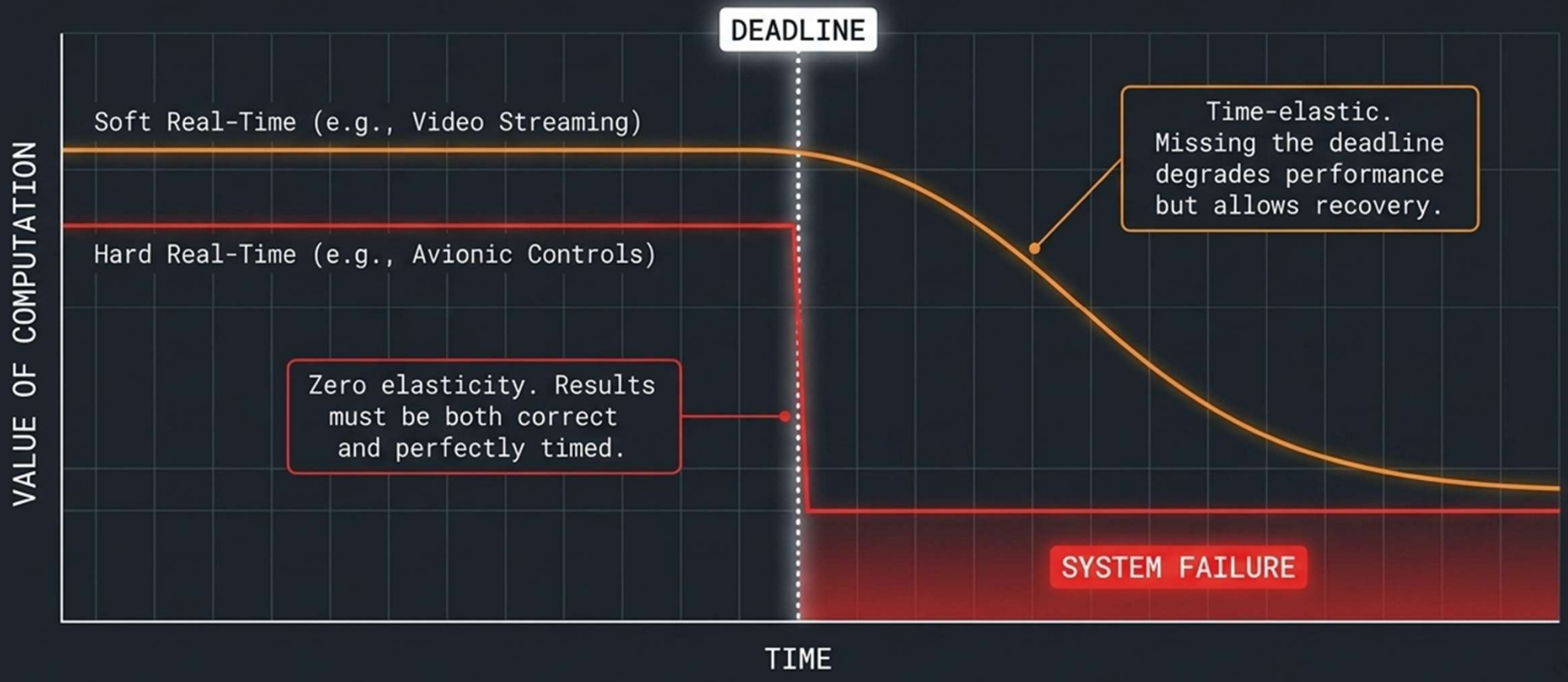


Cyber-Physical Systems (CPS) consist of hardware controllers acting within mechanical or electronic environments.

Standard computing paradigms treat time as a flexible resource. CPS requires absolute time determinism to operate safely.

NotebookLM

The ‘Catastrophic Cliff’ separates hard and soft real-time.



NotebookLM

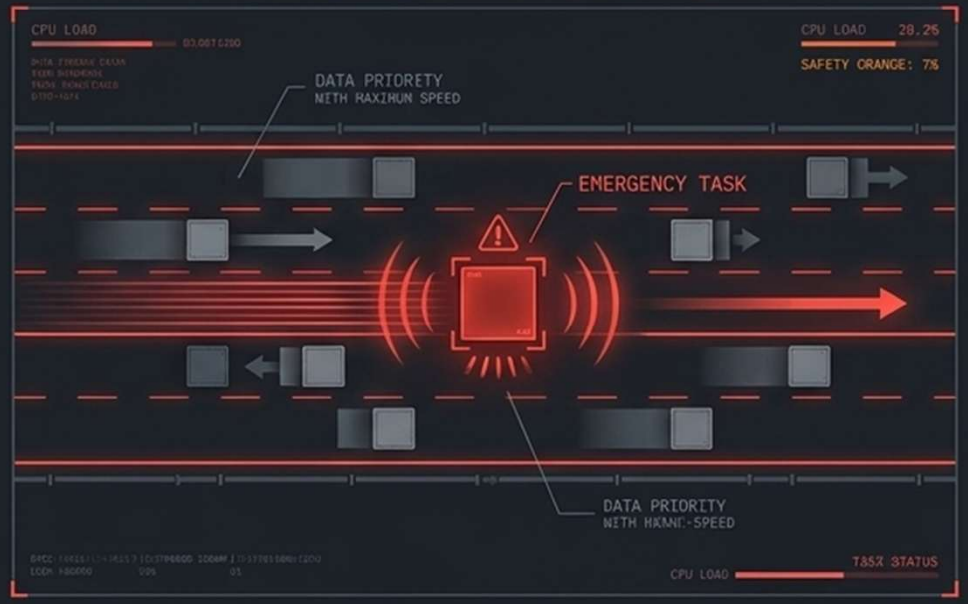
The fairness fallacy of standard computing.

General Purpose OS (GPOS)

Round-Robin & Fairness

GPOS shares the processor equally. Unpredictable dispatch latencies mean a critical task must wait its turn.

Real-Time OS (RTOS)



Strict Priority & Preemption

High-priority tasks seize the processor instantly and run to completion without interruption.

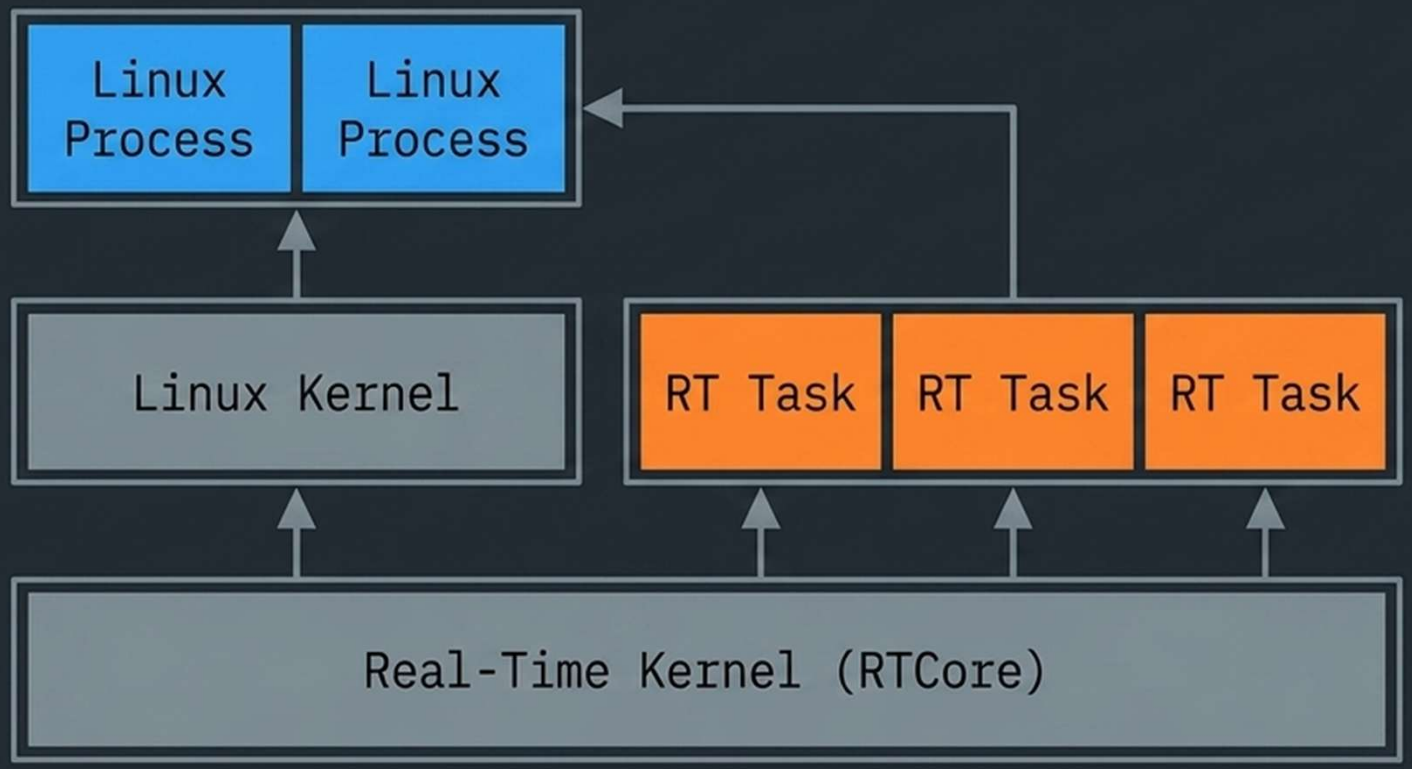
NotebookLM

The OS paradigm shift.

Architecture Dimension	GPOS Paradigm	RTOS Paradigm
Scheduling Policy	Fairness / Round-Robin / FIFO	Priority-based preemptive / EDF / Rate-monotonic
Dispatch Latency	Unpredictable; cannot cancel lower-priority system calls	Guaranteed upper bounds on interrupt disabling
Kernel Design	Large, monolithic, loaded with extensive APIs	Minimalist micro-kernel handling only signals, timers, scheduler
Failure Tolerance	Interdependent architecture prone to system-wide crashes	Isolated process loading; file system faults do not corrupt kernel

NotebookLM

The RTLinux workaround: Dueling kernels.



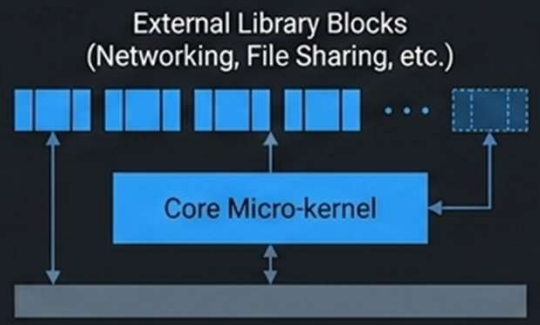
RTCore intercepts hardware interrupts before the standard Linux kernel sees them.

The standard Linux kernel is treated as just another low-priority task that can be preempted at any time.

NotebookLM

The micro-kernel contenders: VxWorks and FreeRTOS.

VxWorks (Commercial)



Design: Single micro-kernel loading external libraries (networking, file sharing) as needed.

Scheduler: Preemptive round-robin (priorities 0-255). Supports POSIX API.

Key Feature: Highly flexible but resource-constrained.

FreeRTOS (Lightweight)

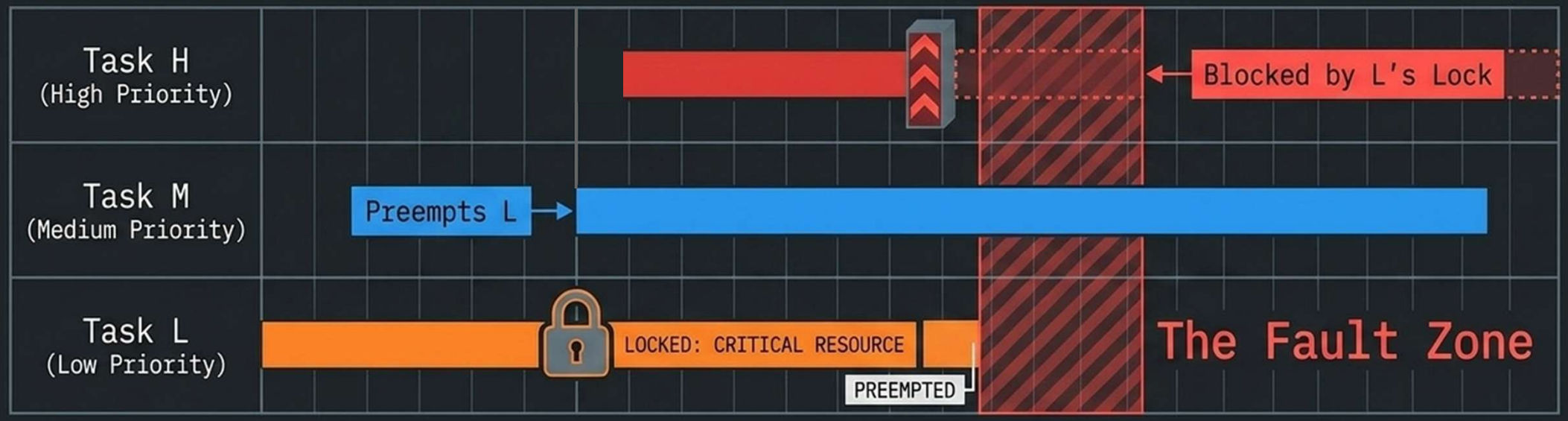
Design: Lightweight micro-kernel supporting dynamic and priority-based scheduling.

Scheduler: Cooperative or preemptive algorithms.

Key Feature: Avoids priority inversion entirely by enforcing non-blocking tasks and fixed-time blocking.

T-00:00:00 T-00:01:00 T-00:02:00 T-00:03:00 T-00:04:00 T+00:05:00 T+00:03:00 T+00:05:00
NotebookLM

The anatomy of a deadlock: Priority Inversion.



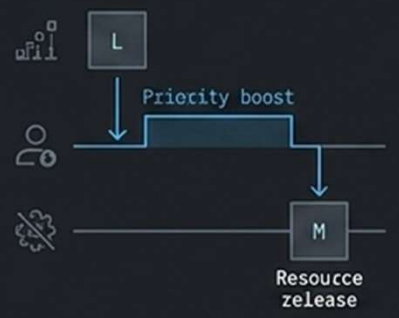
A medium-priority task inadvertently holds a mission-critical task hostage by preventing the low-priority task from releasing the lock.

NotebookLM

Algorithmic antidotes to stolen time.

1. Priority Inheritance

Task L temporarily “inherits” Task H’s high priority until the resource is released, preventing Task M from interrupting.

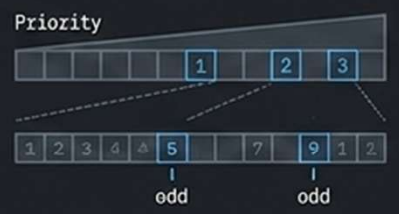


2. Priority Ceiling

The shared resource itself is assigned a strict priority equal to the highest task that might ever lock it.

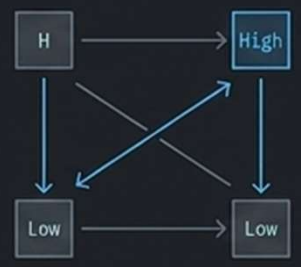
3. Priority Remapping

Expands available priorities from 64 to 128 (multiplying by 2). Uses the newly created “odd” priorities as hidden inversion markers.



4. Priority Exchange

Temporarily swaps the priorities of the blocked high task and the blocking low task, swapping back once the critical section is clear.

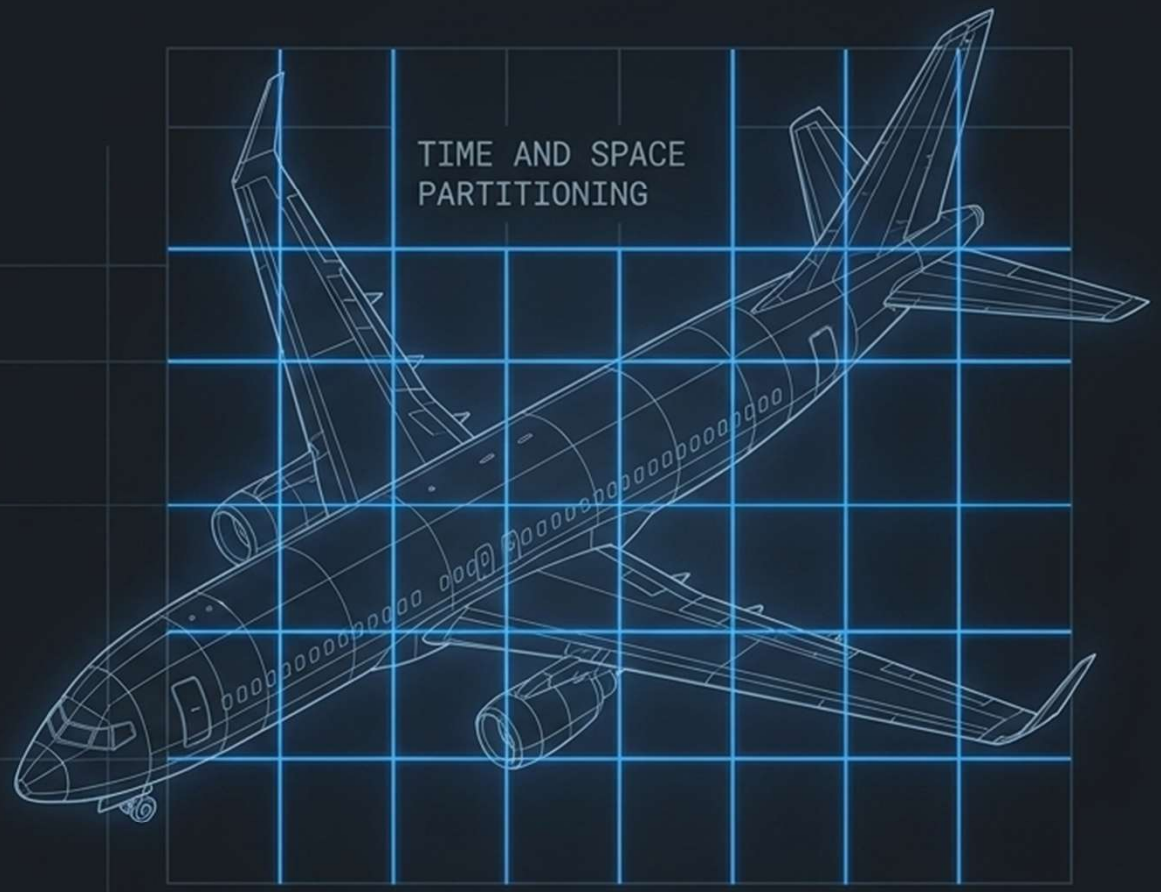


NotebookLM

Proven in the air: ARINC653 and JetOS.

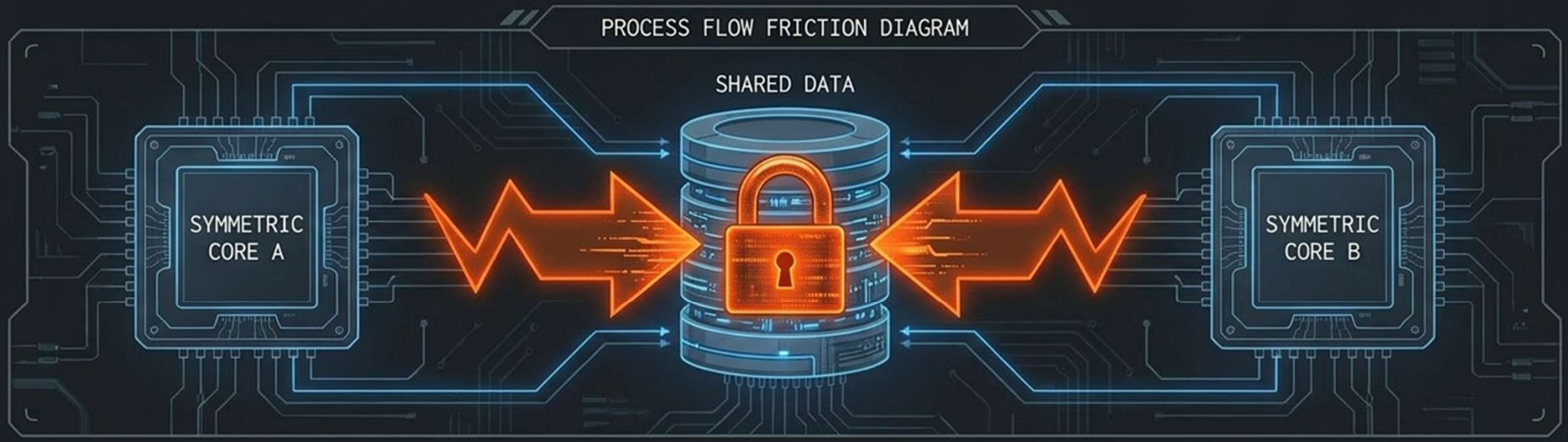
Avionics demands absolute adherence to the ARINC653 international safety standard, enforcing strict isolation of system processes.

JetOS (built on x86/PowerPC) was engineered specifically to meet this standard by enforcing absolute separation of memory and computing resources.



NotebookLM

The future horizon: The multicore bottleneck.



General computing uses symmetric multi-processing (SMP), where all cores run the exact same kernel code.

To prevent data corruption, SMP architecture relies heavily on fine-grained locking mechanisms.

In hard real-time environments, waiting for a lock introduces unpredictable latency-destroying determinism. We cannot simply "add more cores".

T+00:21:00 T+00:21:00 T+00:21:00 T+00:12:00 T+00:13:00 T+00:14:00 T+00:15:00 T+00:16:00 T+00:15:00 T+00:19:00 T+00:23:00 T+00:23:00 NotebookLM

Asymmetric orchestration: The HIPPEROS micro-kernel.

THE MASTER CORE

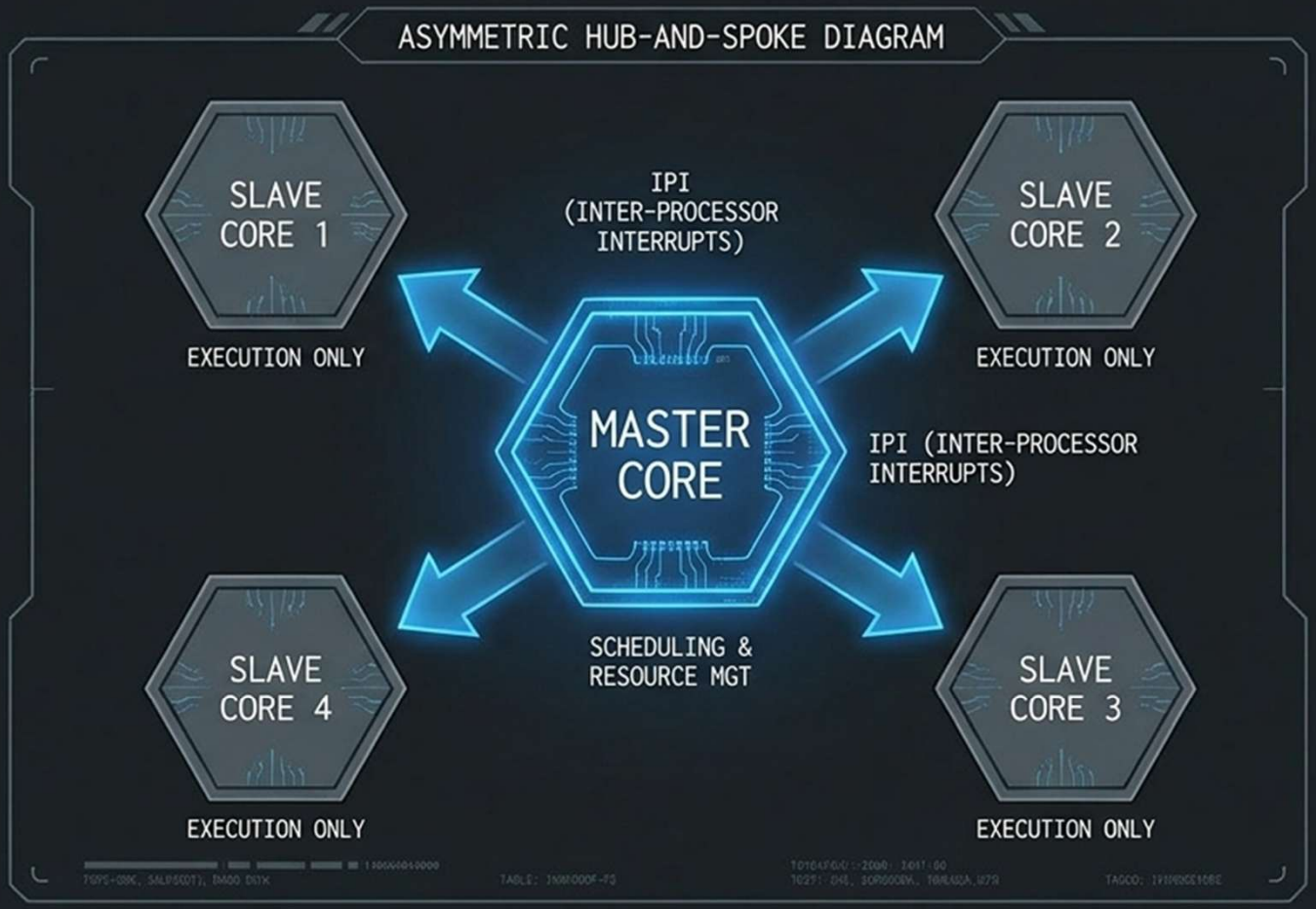
Dedicated exclusively to executing system calls, managing resources, and making all scheduling decisions.

THE SLAVE CORES

Execute parallel context switches only when explicitly notified by the Master via an IPI.

THE RESULT

Pure parallel processing for up to 8 cores without a single fine-grained lock mechanism slowing down the system.



T+08:25:00 T+00:26:00 T+00:21:00 T+08:12:00 T+00:13:00 T+00:14:00 T+08:15:00 T+00:16:00 T+00:15:00 T+08:19:00 T+00:21:00 T+00:23:00 NotebookLM

The expanding attack surface.

Open Debug Ports

Remote memory dumps and unauthorized function calls (e.g., US-Cert VU362332 vulnerability in VxWorks).

Payload Attacks

Modifying PLC control programs to inflict catastrophic physical damage (the exact attack vector utilized by Stuxnet).

Timing Alteration

Injecting malicious code that silently alters the Worst-Case Execution Time (WCET) of a mission-crition-critical system.

T+08:27:00 T+08:28:00 T+08:29:00 T+08:30:00 T+08:32:00 T+08:34:00 T+08:35:00 T+08:26:00 T+08:27:00 T+08:28:00

NotebookLM

Defense via determinism.

Directive 1:

How do we secure cyber-physical systems? We weaponize their predictability.

Directive 2:

Because an RTOS maps absolute worst-case execution times (WCET) and exact system call sequences, any micro-deviation in timing or power consumption instantly exposes an attack.

Directive 3:

True mastery of the cyber-physical domain requires perfect control over both time and trust.

T+00:00:00 T+00:00:01 T+08:00:02 T+08:00:02 T+08:00:03 T+08:34:00 T+08:35:00 T+08:26:00 T+08:27:00 T+08:28:00

NotebookLM

Reference

A. Serino and L. Cheng, "Real-Time Operating Systems for Cyber-Physical Systems: Current Status and Future Research," 2020 International Conferences on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData) and IEEE Congress on Cybermatics (Cybermatics), Rhodes, Greece, 2020, pp. 419-425, doi: 10.1109

T+00:21:00 T+00:21:00 T+00:21:00 T+00:12:00 T+00:13:00 T+00:14:00 T+00:15:00 T+00:16:00 T+00:15:00 T+00:19:00 T+00:23:00 T+00:23:00 NotebookLM

Thank you, Any questions?

T+00:21:00 T+00:21:00 T+00:21:00 T+00:12:00 T+00:13:00 T+00:14:00 T+00:15:00 T+00:16:00 T+00:15:00 T+00:19:00 T+00:23:00 T+00:23:00 NotebookLM